Namal University Mianwali

Department of Computer Science

Vehicle Transport Management System

Prepared by:

Name	Roll No
Tahira Hanif	NUM-BSCS2024-76
Momina Umar	NUM-BSCS2024-36
Mubashir Aman	NUM-BSCS2024-37

Course: Database
Instructor: Asiya Batool

Date: June 16, 2026

Contents

CHAPTER 1 PROJECT OVERVIEW

1.1 INTRODUCTION

Efficient vehicle transportation is essential for businesses involved in vehicle sales services. Managing shipments, assigning drivers and transport trucks, tracking deliveries, handling payments, and maintaining customer records are critical activities that directly affect operational efficiency and customer satisfaction. However, many organizations still rely on manual processes and disconnected record-keeping systems, which can lead to delays, data inconsistencies, poor coordination, and limited visibility into transportation operations.

To address these challenges, this project proposes a Vehicle Transportation Management System (VTMS), a web-based software solution that automates and centralizes vehicle transportation operations. The system enables businesses to manage shipments, assign drivers and trucks, track delivery progress, process payments, generate invoices, and verify deliveries through a single platform. VTMS provides a secure, scalable, and efficient solution that improves operational control, enhances transparency, and ensures timely vehicle delivery to customers.

1.2 PROBLEM STATEMENT

Vehicle sellers in Pakistan often face difficulties arranging reliable and timely transportation for their vehicles. Existing transport services do not have a centralized system to manage shipments efficiently. This lack of coordination causes delays, inaccurate fee calculations, and limited transparency. Customers cannot easily track their vehicles in real time or verify deliveries securely. Payments and refunds are often unclear or handled inconsistently. Transportation companies also face challenges. They struggle to manage available trucks and drivers. Coordinating shipment assignments and maintaining accurate records of vehicles, payments, and customer information is difficult. These issues lead to missed shipments, underused resources, and low customer satisfaction. The Vehicle Transport Management System (VTMS) will solve these problems. The system allows customers to book shipments, calculate fees automatically, and make payments securely. It verifies deliveries using OTP and tracks shipments in real time. The system also manages trucks and drivers efficiently. It stores accurate data and provides

clear records. VTMS improves transparency, reliability, and the overall efficiency of domestic vehicle transportation services. “‘latex

1.3 PROJECT OBJECTIVES

The primary objective of this project is to design and develop a Vehicle Transportation Management System (VTMS) that enables businesses to efficiently manage the transportation of vehicles to their customers. The system aims to automate transportation operations, improve coordination among stakeholders, provide shipment visibility, and reduce the inefficiencies associated with manual processes.

The specific objectives of the VTMS are as follows:

- Provide a centralized platform for managing vehicle transportation operations, including shipment creation, tracking, delivery verification, and payment processing.
- Enable businesses to register customers, create transportation requests, and maintain complete records of transportation activities.
- Facilitate efficient assignment and management of drivers and transport trucks to ensure timely delivery of vehicles.
- Provide real-time shipment tracking through unique tracking identifiers, allowing businesses and customers to monitor delivery progress.
- Automate transportation charge calculation using route and distance information obtained through the OpenRouteService API.
- Generate invoices, process payments, and maintain accurate financial records related to transportation services.
- Improve operational efficiency by reducing manual paperwork, minimizing data redundancy, and streamlining transportation workflows.
- Implement Role-Based Access Control (RBAC) to ensure that users can access only the functionalities and information relevant to their responsibilities.
- Maintain secure, accurate, and consistent information through a centralized database management system.
- Support informed decision-making by providing transportation records, shipment history, payment information, and operational reports.

GitHub Repository: `https://github.com/MominaUmar74/VTMS.git`

CHAPTER 2 CONCEPTUAL DATABASE DESIGN

2.1 ENTITY

The following table identifies and defines all entities in the VTMS database. Each entity represents a distinct real world concept central to the vehicle transportation workflow. Associative entities arising from many-to-many relationships (such as Driver_Assignment) are included here as they appear as named tables in the ERD.

Sr. No	Entity Name	Description
01	Company	The Company entity represents transportation companies registered within the system. It stores company-related information including registration number, contact details, address, wallet balance, and operational status. Companies can create agreements, receive invoices, and process payments through the system.
02	User	The User entity represents all authenticated individuals who can access the Vehicle Transportation Management System. Users may have different roles such as administrator, Transport manager,operational staff. The entity stores login credentials, personal information, account status, and role details required for authentication and authorization.
03	Driver	An employee responsible for operating an assigned truck and delivering vehicles. Extends User with license and status details.
04	Vehicle	The Vehicle entity stores detailed information about vehicles used for transportation operations. It includes registration number, manufacturer details, model, year, color, category, and current operational status. Vehicles are assigned to shipments according to transportation requirements.

05	Vehicle_Category	The VehicleCategory entity classifies vehicles into different categories based on capacity, pricing, and transportation requirements. It defines base transportation rates and pricing multipliers used for shipment cost calculations. This entity helps standardize vehicle classification across the system.
06	Shipment	The Shipment entity is the central entity of the system and represents transportation orders created by companies. It contains shipment details such as pickup and delivery locations, distance, estimated charges, shipment status, assigned vehicle, customer information, and associated agreements.
07	Truck	The Truck entity represents transportation trucks available for shipment operations. It stores truck-specific information such as truck type, carrying capacity, and operational status. Trucks are assigned to drivers and shipments through the driver assignment process.
08	Driver_Assignment	An associative entity recording which driver and truck are assigned to a shipment, including assignment date, end date, and assignment status.
09	Shipment_Status_Log	An audit trail table that records every status change for a shipment, including old status, new status, who changed it, timestamp, notes, and current location.
10	Payment	The Payment entity records all financial transactions related to shipment services. It stores payment amounts, payment methods, payment dates, payment status, and associated company or shipment information. This entity supports financial accountability and transaction tracking.
11	EndCustomer	The EndCustomer entity represents customers who request transportation services from a company. It stores customer contact information, address details, and identification data. End customers are associated with shipments and enable companies to manage transportation requests effectively.

12	DeliveryVerification	The DeliveryVerification entity confirms successful shipment delivery. It records verification details such as pickup verification status, delivery verification status, verification timestamps, and responsible personnel. This entity ensures shipment completion authenticity and accountability.
13	ShipmentStatusLog	The ShipmentStatusLog entity maintains a historical record of all shipment status changes throughout the shipment lifecycle. It stores status updates, timestamps, remarks, and user information responsible for the changes. This entity supports auditing, tracking, and shipment monitoring.
14	CancellationRequest	The CancellationRequest entity manages requests to cancel assigned shipments. It stores cancellation reasons, approval information, request timestamps, approval status, and associated shipment details. This entity ensures controlled and authorized cancellation of transportation operations.

Table 2.1: VTMS Entity Definitions

2.2 DATA DICTIONARY

The data dictionary below defines every attribute of each entity identified in the ERD, including data type, constraint, and description. Foreign Keys (FK) are noted in the attribute name and description.

2.2.1 User

Represents all authenticated users of the Vehicle Transportation Management System. Users may perform different operations based on their assigned roles such as Administrator, Manager, Dispatcher, or Operational Staff. The entity stores authentication credentials and profile information required for system access and role-based authorization.

Sr. No	Name	Data Type	Constraint	Description
01	user_id (PK)	BIGINT	NOT NULL, UNIQUE	Unique identifier for each system user.

02	username	VARCHAR(100)	NOT NULL, UNIQUE	Username used for authentication.
03	password_hash	VARCHAR(255)	NOT NULL	Encrypted password stored securely.
04	full_name	VARCHAR(100)	NOT NULL	Complete name of the user.
05	email	VARCHAR(100)	NOT NULL, UNIQUE	User email address.
06	role	ENUM	NOT NULL	Assigned role within the system.
07	status	VARCHAR(20)	NOT NULL	Current account status.
08	created_at	DATETIME	NOT NULL	Account creation timestamp.

2.2.2 Driver

Represents vehicle operators responsible for transporting shipments. Drivers are registered users with valid licenses and availability information used during shipment assignment.

Sr. No	Name	Data Type	Constraint	Description
01	driver_id (PK)	BIGINT	NOT NULL, UNIQUE	Unique identifier of the driver.
02	user_id	BIGINT	NOT NULL	Associated user account identifier.
03	license_no	VARCHAR(50)	NOT NULL, UNIQUE	Driver's license number.
04	current_status	VARCHAR(20)	NOT NULL	Availability status of the driver.

2.2.3 Company

Represents transportation companies registered in the system. Companies create shipments, enter agreements, receive invoices, and manage payments.

Sr. No	Name	Data Type	Constraint	Description
01	company_id (PK)	BIGINT	NOT NULL, UNIQUE	Unique company identifier.
02	company_name	VARCHAR(150)	NOT NULL	Registered company name.
03	registration_number	VARCHAR(100)	NOT NULL, UNIQUE	Government registration number.

04	email	VARCHAR(100)	NOT NULL	Official company email.
05	phone	VARCHAR(20)	NOT NULL	Company contact number.
06	address	VARCHAR(255)	NOT NULL	Registered business address.
07	wallet_balance	DECIMAL(12,2)	DEFAULT 0	Available wallet balance.
08	status	VARCHAR(20)	NOT NULL	Operational status of company.

2.2.4 EndCustomer

Represents customers who request transportation services from companies. Their information is used for shipment booking and communication.

Sr. No	Name	Data Type	Constraint	Description
01	end_customer_id (PK)	BIGINT	NOT NULL, UNIQUE	Unique identifier of customer.
02	full_name	VARCHAR(100)	NOT NULL	Customer's complete name.
03	phone	VARCHAR(20)	NOT NULL	Customer contact number.
04	email	VARCHAR(100)	NULL	Customer email address.
05	address	VARCHAR(255)	NOT NULL	Customer residential/business address.
06	id_proof_type	VARCHAR(50)	NOT NULL	Type of identification document.
07	id_proof_number	VARCHAR(100)	NOT NULL, UNIQUE	Identification document number.

2.2.5 VehicleCategory

Represents classifications of vehicles used to determine transportation pricing and operational capabilities.

Sr. No	Name	Data Type	Constraint	Description
01	category_id (PK)	BIGINT	NOT NULL, UNIQUE	Unique category identifier.
02	category_name	VARCHAR(100)	NOT NULL	Vehicle category name.
03	base_price_per_km	DECIMAL(10,2)	NOT NULL	Base transportation rate per kilometer.
04	multiplier	DECIMAL(5,2)	NOT NULL	Pricing multiplier for category.

2.2.6 Vehicle

Represents vehicles available for shipment transportation. Each vehicle belongs to a category and can be assigned to shipments.

Sr. No	Name	Data Type	Constraint	Description
01	vehicle_id (PK)	BIGINT	NOT NULL, UNIQUE	Unique vehicle identifier.
02	registration_number	VARCHAR(50)	NOT NULL, UNIQUE	Official registration number.
03	make	VARCHAR(50)	NOT NULL	Vehicle manufacturer.
04	model	VARCHAR(50)	NOT NULL	Vehicle model.
05	year	INT	NOT NULL	Manufacturing year.
06	color	VARCHAR(30)	NULL	Vehicle color.
07	category_id	BIGINT	NOT NULL	Vehicle category reference.
08	status	VARCHAR(20)	NOT NULL	Current vehicle status.

2.2.7 Truck

Represents trucks available for logistics operations. Trucks are assigned to drivers and shipments based on transportation requirements.

Sr. No	Name	Data Type	Constraint	Description
01	truck_id (PK)	BIGINT	NOT NULL, UNIQUE	Unique truck identifier.
02	truck_type	VARCHAR(50)	NOT NULL	Type of truck.
03	capacity	DECIMAL(10,2)	NOT NULL	Cargo carrying capacity.
04	current_status	VARCHAR(20)	NOT NULL	Current operational status.

2.2.8 Shipment

Represents transportation orders created by companies for moving goods from a pickup location to a delivery destination. It serves as the central entity of the VTMS and maintains all operational shipment information.

Sr. No	Name	Data Type	Constraint	Description
01	shipment_id (PK)	BIGINT	NOT NULL, UNIQUE	Unique identifier for each shipment.

02	company_id	BIGINT	NOT NULL	Company responsible for the shipment.
03	end_customer_id	BIGINT	NOT NULL	Customer associated with the shipment.
04	vehicle_id	BIGINT	NOT NULL	Vehicle assigned to the shipment.
05	agreement_id	BIGINT	NOT NULL	Agreement governing shipment terms.
06	tracking_id	VARCHAR(100)	NOT NULL, UNIQUE	Shipment tracking identifier.
07	pickup_location	VARCHAR(255)	NOT NULL	Origin location of shipment.
08	delivery_location	VARCHAR(255)	NOT NULL	Destination location of shipment.
09	total_distance_km	DECIMAL(10,2)	NOT NULL	Distance calculated through OpenRouteService API.
10	total_charges	DECIMAL(12,2)	NOT NULL	Estimated transportation charges.
11	total_amount	DECIMAL(12,2)	NOT NULL	Final amount payable for shipment.
12	shipment_status	VARCHAR(30)	NOT NULL	Current shipment status.
13	created_at	DATETIME	NOT NULL	Shipment creation timestamp.

2.2.9 DriverAssignment

Represents assignment records linking drivers and trucks with shipments. It ensures efficient allocation of transportation resources.

Sr. No	Name	Data Type	Constraint	Description
01	assignment_id (PK)	BIGINT	NOT NULL, UNIQUE	Unique assignment identifier.
02	shipment_id	BIGINT	NOT NULL	Assigned shipment identifier.
03	driver_id	BIGINT	NOT NULL	Assigned driver identifier.
04	truck_id	BIGINT	NOT NULL	Assigned truck identifier.
05	assigned_date	DATE	NOT NULL	Date of assignment.
06	end_date	DATE	NULL	Assignment completion date.
07	status	VARCHAR(20)	NOT NULL	Current assignment status.

2.2.10 Agreement

Represents contractual agreements between transportation companies and service providers. Agreements define pricing structures and business terms.

Sr. No	Name	Data Type	Constraint	Description
01	agreement_id (PK)	BIGINT	NOT NULL, UNIQUE	Unique agreement identifier.
02	company_id	BIGINT	NOT NULL	Company involved in agreement.
03	created_by	BIGINT	NOT NULL	User who created the agreement.
04	agreement_type	VARCHAR(50)	NOT NULL	Type of transportation agreement.
05	start_date	DATE	NOT NULL	Agreement start date.
06	end_date	DATE	NOT NULL	Agreement expiry date.
07	base_price_per_km	DECIMAL(10,2)	NOT NULL	Base transportation price.
08	discount_percentage	DECIMAL(5,2)	NULL	Discount percentage applied.
09	status	VARCHAR(20)	NOT NULL	Current agreement status.

2.2.11 Invoice

Represents invoices generated for shipment transactions. It provides billing information and supports financial management.

Sr. No	Name	Data Type	Constraint	Description
01	invoice_id (PK)	BIGINT	NOT NULL, UNIQUE	Unique invoice identifier.
02	company_id	BIGINT	NOT NULL	Company receiving invoice.
03	shipment_id	BIGINT	NOT NULL	Related shipment identifier.
04	invoice_number	VARCHAR(100)	NOT NULL, UNIQUE	Official invoice number.
05	total_amount	DECIMAL(12,2)	NOT NULL	Invoice total amount.
06	status	VARCHAR(20)	NOT NULL	Current invoice status.
07	generated_at	DATETIME	NOT NULL	Invoice generation timestamp.

2.2.12 Payment

Represents financial transactions performed against shipment invoices. It maintains payment history and transaction status.

Sr. No	Name	Data Type	Constraint	Description
01	payment_id (PK)	BIGINT	NOT NULL, UNIQUE	Unique payment identifier.
02	shipment_id	BIGINT	NOT NULL	Shipment associated with payment.
03	company_id	BIGINT	NOT NULL	Paying company identifier.
04	amount	DECIMAL(12,2)	NOT NULL	Amount paid.
05	payment_type	VARCHAR(30)	NOT NULL	Method of payment.
06	payment_date	DATE	NOT NULL	Date payment was made.
07	payment_status	VARCHAR(20)	NOT NULL	Current payment status.
08	processed_by	BIGINT	NOT NULL	User who processed payment.
09	paid_at	DATETIME	NOT NULL	Timestamp of payment processing.

2.2.13 DeliveryVerification

Represents confirmation records used to verify successful pickup and delivery of shipments.

Sr. No	Name	Data Type	Constraint	Description
01	verification_id (PK)	BIGINT	NOT NULL, UNIQUE	Unique verification identifier.
02	shipment_id	BIGINT	NOT NULL	Shipment being verified.
03	pickup_verified	BOOLEAN	NOT NULL	Pickup verification status.
04	delivery_verified	BOOLEAN	NOT NULL	Delivery verification status.
05	verified_by	VARCHAR(100)	NOT NULL	Person responsible for verification.
06	verified_at	DATETIME	NOT NULL	Verification timestamp.

2.2.14 ShipmentStatusLog

Represents the complete history of shipment status updates. It provides auditing and shipment tracking capabilities.

Sr. No	Name	Data Type	Constraint	Description
01	log_id (PK)	BIGINT	NOT NULL, UNIQUE	Unique log identifier.
02	shipment_id	BIGINT	NOT NULL	Shipment associated with status update.
03	old_status	VARCHAR(30)	NOT NULL	Previous shipment status.
04	new_status	VARCHAR(30)	NOT NULL	Updated shipment status.
05	changed_by	BIGINT	NOT NULL	User who changed status.
06	remarks	VARCHAR(255)	NULL	Additional status remarks.
07	created_at	DATETIME	NOT NULL	Status update timestamp.

2.2.15 CancellationRequest

Represents requests submitted for shipment cancellation. It records approval workflow and cancellation reasons.

Sr. No	Name	Data Type	Constraint	Description
01	request_id (PK)	BIGINT	NOT NULL, UNIQUE	Unique cancellation request identifier.
02	shipment_id	BIGINT	NOT NULL	Shipment requested for cancellation.
03	requested_by	BIGINT	NOT NULL	User who submitted cancellation request.
04	approved_by	BIGINT	NULL	User who approved cancellation.
05	reason	VARCHAR(500)	NOT NULL	Reason provided for cancellation.
06	request_status	VARCHAR(20)	NOT NULL	Approval status of request.
07	requested_at	DATETIME	NOT NULL	Request submission timestamp.
08	resolved_at	DATETIME	NULL	Request resolution timestamp.

2.3 BUSINESS RULES

The following business rules define the conditions, constraints, and operational logic that govern the Vehicle Transportation Management System (VTMS). These rules ensure data con-

sistency, integrity, security, and efficient transportation operations across all modules of the system.

2.3.1 Company and Customer Management Rules

1. A Company can register multiple End Customers, while each End Customer belongs to exactly one Company.
2. Every Company must possess a unique Registration Number before it can create shipments or agreements.
3. A Company account must be in Active status before any shipment booking can be processed.
4. An End Customer must provide valid contact information, including phone number and address, before a shipment can be created.
5. Each End Customer must have a unique identification document number within the system.

2.3.2 Agreement Management Rules

1. A Company may have multiple Agreements, but each Agreement belongs to exactly one Company.
2. Every Shipment must be associated with a valid and active Agreement.
3. The Agreement Start Date must be earlier than the Agreement End Date.
4. Expired Agreements cannot be used for creating new Shipments.
5. Discount percentages defined within an Agreement must remain between 0

2.3.3 Vehicle and Fleet Management Rules

1. Every Vehicle must belong to exactly one Vehicle Category.
2. Vehicle Registration Numbers must be unique throughout the system.
3. Vehicles marked as Inactive or Maintenance cannot be assigned to new Shipments.
4. Each Vehicle Category must define a valid Base Price Per Kilometer and Pricing Multiplier.

5. A Vehicle can participate in multiple Shipments over time but may only be assigned to one active Shipment at a time.

2.3.4 Driver and Truck Assignment Rules

1. A Driver must possess a valid driving license before assignment to any Shipment.
2. Only Drivers with Current Status = Available may be assigned to Shipments.
3. Only Trucks with Current Status = Available may be assigned to Shipments.
4. Each Driver Assignment must reference one Driver, one Truck, and one Shipment.
5. A Driver cannot be assigned to multiple active Shipments simultaneously.
6. A Truck cannot be assigned to multiple active Shipments simultaneously.
7. When an assignment is completed, the Driver and Truck statuses must automatically revert to Available.

2.3.5 Shipment Management Rules

1. Every Shipment must belong to one Company and one End Customer.
2. Every Shipment must have a unique Tracking ID.
3. Pickup Location and Delivery Location must not be identical.
4. Total Distance must be greater than zero before a Shipment can be confirmed.
5. Shipment charges must be calculated before Shipment approval.
6. A Shipment cannot be marked as Delivered until Delivery Verification has been completed.
7. Shipment information cannot be deleted after payment processing has begun.

2.3.6 Route Calculation Rules

1. Transportation distance must be calculated using the OpenRouteService API.
2. The calculated route distance must be stored in the Shipment record.
3. Shipment charges must be based on the calculated distance and applicable pricing rules.

4. If the OpenRouteService API fails to return a valid route, the Shipment cannot proceed to confirmation.
5. Route calculations must be refreshed whenever pickup or delivery locations are modified.

2.3.7 Invoice and Billing Rules

1. An Invoice may only be generated for an approved Shipment.
2. Each Invoice must reference exactly one Shipment.
3. Invoice Numbers must be unique throughout the system.
4. Invoice amounts must match the final calculated Shipment charges.
5. An Invoice cannot be marked as Paid unless a corresponding Payment record exists.

2.3.8 Payment Rules

1. Every Payment must reference one Company and one Shipment.
2. Payment Amount must be greater than zero.
3. Payment records cannot be deleted after successful processing.
4. Payment Status must accurately reflect the transaction outcome.
5. Only authorized users may process payment transactions.
6. A Shipment may not be marked as financially completed until payment has been verified.

2.3.9 Delivery Verification Rules

1. Every completed Shipment must have one Delivery Verification record.
2. Pickup Verification must occur before Delivery Verification.
3. Delivery Verification must contain the verification timestamp.
4. Only authorized personnel may perform shipment verification.
5. A Shipment cannot transition to Delivered status unless Delivery Verification has been completed successfully.

2.3.10 Shipment Status Tracking Rules

1. Every Shipment status change must be recorded in ShipmentStatusLog.
2. Shipment statuses must follow the sequence: Pending $\rightarrow$ Approved $\rightarrow$ Assigned $\rightarrow$ In Transit $\rightarrow$ Delivered.
3. Shipment statuses cannot skip intermediate stages.
4. Each status update must record the responsible user and timestamp.
5. Historical Shipment Status Log records must never be deleted.

2.3.11 Cancellation Request Rules

1. Cancellation Requests must reference a valid Shipment.
2. A cancellation request must include a reason for cancellation.
3. Only authorized users may approve or reject cancellation requests.
4. Delivered Shipments cannot be cancelled.
5. Cancellation requests must store request and resolution timestamps.
6. Once approved, the Shipment status must be updated accordingly.

2.3.12 Role-Based Access Control (RBAC) Rules

1. Administrators have full access to all system modules and data.
2. Company Managers may manage Company, Customer, Shipment, Agreement, Invoice, and Payment records belonging to their company.
3. Drivers may access only their assigned Shipment and Assignment records.
4. Finance personnel may process Payments and manage Invoices but cannot modify Shipment assignments.
5. Operational staff may update Shipment statuses and Delivery Verification records.
6. Users may only access system functions permitted by their assigned roles.
7. Every action performed within the system must be traceable to an authenticated User account.

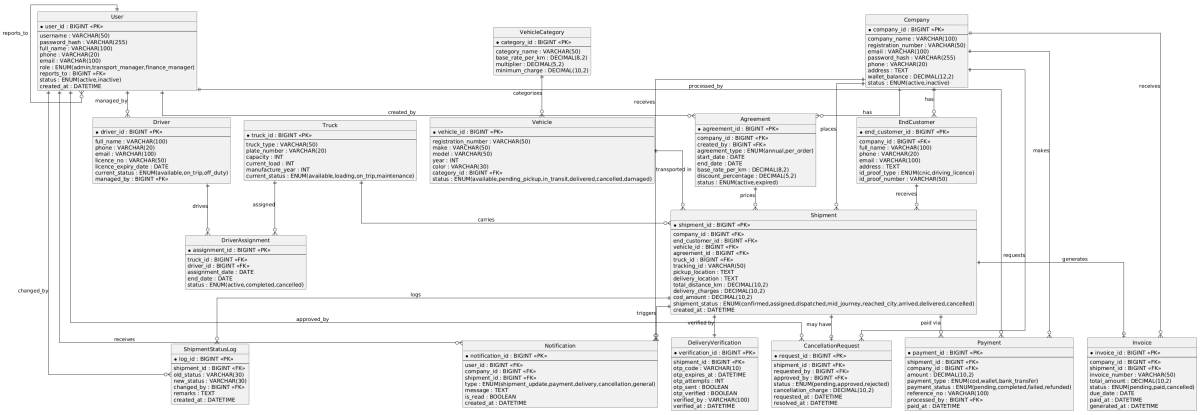


Figure 2.1: Entity relationship Diagram

2.4 ENTITY RELATIONSHIP DIAGRAM

The Entity Relationship Diagram (ERD) below was created using draw.io (diagrams.net) and illustrates all entities, attributes, primary keys, foreign keys, relationships, and cardinalities within the VTMS database. All entity names, attribute names, and relationship lines are taken directly from the submitted ERD file.

CHAPTER 3 LOGICAL DATABASE DESIGN

This chapter presents the logical design of the Vehicle Transportation Management System (VTMS) database. The conceptual Entity Relationship Diagram (ERD) developed in the previous chapter is transformed into a relational schema suitable for implementation in a relational database management system. The chapter also identifies functional dependencies for each relation and demonstrates the normalization process to achieve Third Normal Form (3NF), ensuring minimal redundancy, improved data integrity, and efficient database operations.

3.1 RELATIONAL SCHEMA

This section presents the transformation of the Entity Relationship Diagram (ERD) into a relational schema suitable for implementation in a relational database management system. Each entity identified during the conceptual design phase has been converted into a relation (table), with primary keys used to uniquely identify records and foreign keys used to establish relationships among entities.

The relational schema preserves all relationships defined in the ERD while ensuring data integrity and consistency. One-to-many relationships are represented through foreign keys, whereas many-to-many relationships are resolved using associative relations. The resulting schema provides a structured foundation for implementing the Vehicle Transportation Management System (VTMS) database.

3.1.1 Transformation of Entities into Relations

Each strong entity identified in the ERD was converted into an individual relation. The primary key of each entity was retained as the primary key of the corresponding relation. The following relations were obtained:

- USER
- DRIVER

- COMPANY
- END_CUSTOMER
- VEHICLE_CATEGORY
- VEHICLE
- TRUCK
- AGREEMENT
- SHIPMENT
- DRIVER_ASSIGNMENT
- INVOICE
- PAYMENT
- DELIVERY_VERIFICATION
- SHIPMENT_STATUS_LOG
- CANCELLATION_REQUEST

3.1.2 Representation of Relationships

The relationships identified in the ERD were converted according to standard relational database design principles:

1. The relationship between COMPANY and END_CUSTOMER is represented using the foreign key `company_id` in the END_CUSTOMER relation.
2. The relationship between VEHICLE_CATEGORY and VEHICLE is represented using the foreign key `category_id` in the VEHICLE relation.
3. The relationship between COMPANY and AGREEMENT is represented using the foreign key `company_id` in the AGREEMENT relation.
4. The relationship between COMPANY and SHIPMENT is represented using the foreign key `company_id` in the SHIPMENT relation.
5. The relationship between END_CUSTOMER and SHIPMENT is represented using the foreign key `end_customer_id` in the SHIPMENT relation.

6. The relationship between AGREEMENT and SHIPMENT is represented using the foreign key `agreement_id` in the SHIPMENT relation.

7. The relationship between VEHICLE and SHIPMENT is represented using the foreign key `vehicle_id` in the SHIPMENT relation.

8. The many-to-many relationship between DRIVER, TRUCK, and SHIPMENT is resolved through the associative relation DRIVER_ASSIGNMENT.

9. The relationship between SHIPMENT and INVOICE is represented using the foreign key `shipment_id` in the INVOICE relation.

10. The relationship between SHIPMENT and PAYMENT is represented using the foreign key `shipment_id` in the PAYMENT relation.

11. The relationship between SHIPMENT and DELIVERY_VERIFICATION is represented using the foreign key `shipment_id` in the DELIVERY_VERIFICATION relation.

12. The relationship between SHIPMENT and SHIPMENT_STATUS_LOG is represented using the foreign key `shipment_id` in the SHIPMENT_STATUS_LOG relation.

13. The relationship between SHIPMENT and CANCELLATION_REQUEST is represented using the foreign key `shipment_id` in the CANCELLATION_REQUEST relation.

3.1.3 Relational Schema

The final relational schema derived from the ERD is presented below.

```
USER(
    user_id PK,
    username,
    password_hash,
    full_name,
    email,
    role,
    status,
    created_at
)

DRIVER(
    driver_id PK,
    user_id FK,
```

```
    license_no,
    current_status
)

COMPANY(
    company_id PK,
    company_name,
    registration_number,
    email,
    phone,
    address,
    wallet_balance,
    status
)

END_CUSTOMER(
    end_customer_id PK,
    company_id FK,
    full_name,
    phone,
    email,
    address,
    id_proof_type,
    id_proof_number
)

VEHICLE_CATEGORY(
    category_id PK,
    category_name,
    base_price_per_km,
    multiplier
)

VEHICLE(
    vehicle_id PK,
    category_id FK,
    registration_number,
    make,
    model,
    year,
```

```
    color,
    status
)

TRUCK(
    truck_id PK,
    truck_type,
    capacity,
    current_status
)

AGREEMENT(
    agreement_id PK,
    company_id FK,
    created_by FK,
    agreement_type,
    start_date,
    end_date,
    base_price_per_km,
    discount_percentage,
    status
)

SHIPMENT(
    shipment_id PK,
    company_id FK,
    end_customer_id FK,
    vehicle_id FK,
    agreement_id FK,
    tracking_id,
    pickup_location,
    delivery_location,
    total_distance_km,
    total_charges,
    total_amount,
    shipment_status,
    created_at
)

DRIVER_ASSIGNMENT(
```

```
    assignment_id PK,
    shipment_id FK,
    driver_id FK,
    truck_id FK,
    assigned_date,
    end_date,
    status
)

INVOICE(
    invoice_id PK,
    company_id FK,
    shipment_id FK,
    invoice_number,
    total_amount,
    status,
    generated_at
)

PAYMENT(
    payment_id PK,
    shipment_id FK,
    company_id FK,
    amount,
    payment_type,
    payment_date,
    payment_status,
    processed_by FK,
    paid_at
)

DELIVERY_VERIFICATION(
    verification_id PK,
    shipment_id FK,
    pickup_verified,
    delivery_verified,
    verified_by,
    verified_at
)
```

```
SHIPMENT_STATUS_LOG(
    log_id PK,
    shipment_id FK,
    old_status,
    new_status,
    changed_by FK,
    remarks,
    created_at
)

CANCELLATION_REQUEST(
    request_id PK,
    shipment_id FK,
    requested_by FK,
    approved_by FK,
    reason,
    request_status,
    requested_at,
    resolved_at
)
```

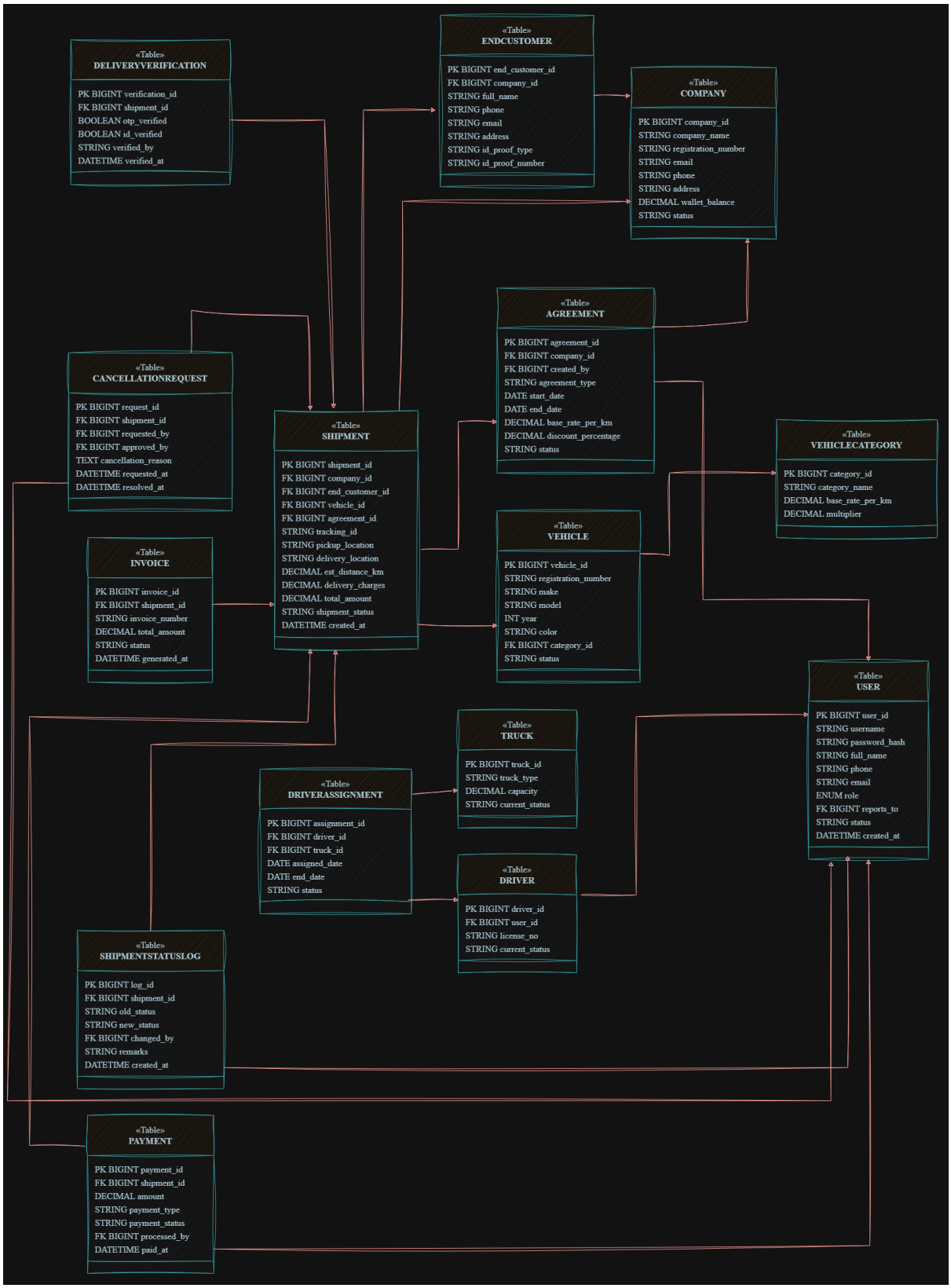


Figure 3.1: Relation Schema

3.2 FUNCTIONAL DEPENDENCIES

Functional dependencies define the relationships among attributes within a relation. They are used to identify candidate keys, remove redundancy, and support the normalization process.

3.2.1 USER

- user_id → username, password_hash, full_name, email, role, status, created_at
- email → user_id
- username → user_id

3.2.2 DRIVER

- driver_id → user_id, license_no, current_status
- license_no → driver_id

3.2.3 COMPANY

- company_id → company_name, registration_number, email, phone, address, wallet_balance, status
- registration_number → company_id

3.2.4 END_CUSTOMER

- end_customer_id → company_id, full_name, phone, email, address, id_proof_type, id_proof_number
- id_proof_number → end_customer_id

3.2.5 VEHICLE_CATEGORY

- category_id → category_name, base_price_per_km, multiplier

3.2.6 VEHICLE

- vehicle_id → category_id, registration_number, make, model, year, color, status
- registration_number → vehicle_id

3.2.7 TRUCK

- truck_id → truck_type, capacity, current_status

3.2.8 AGREEMENT

- agreement_id → company_id, created_by, agreement_type, start_date, end_date, base_price_per_km, discount_percentage, status

3.2.9 SHIPMENT

- shipment_id → company_id, end_customer_id, vehicle_id, agreement_id, tracking_id, pickup_location, delivery_location, total_distance_km, total_charges, total_amount, shipment_status, created_at
- tracking_id → shipment_id

3.2.10 DRIVER_ASSIGNMENT

- assignment_id → shipment_id, driver_id, truck_id, assigned_date, end_date, status

3.2.11 INVOICE

- invoice_id → company_id, shipment_id, invoice_number, total_amount, status, generated_at
- invoice_number → invoice_id

3.2.12 PAYMENT

- payment_id → shipment_id, company_id, amount, payment_type, payment_date, payment_status, processed_by, paid_at

3.2.13 DELIVERY_VERIFICATION

- verification_id → shipment_id, pickup_verified, delivery_verified, verified_by, verified_at

3.2.14 SHIPMENT_STATUS_LOG

- log_id → shipment_id, old_status, new_status, changed_by, remarks, created_at

3.2.15 CANCELLATION_REQUEST

- request_id → shipment_id, requested_by, approved_by, reason, request_status, requested_at, resolved_at

3.3 NORMALIZATION

Normalization is the process of organizing data within a relational database to reduce redundancy, eliminate undesirable insertion, update, and deletion anomalies, and improve overall data integrity. The Vehicle Transportation Management System (VTMS) database was analyzed and normalized up to Third Normal Form (3NF). The normalization process ensures efficient data storage while preserving all required business information and relationships.

3.3.1 First Normal Form (1NF)

A relation is in First Normal Form (1NF) if:

- All attributes contain atomic values.
- No repeating groups exist.
- Each row can be uniquely identified by a primary key.

The VTMS database satisfies First Normal Form because all entities contain atomic attributes and each table possesses a unique primary key. Multi-valued attributes and repeating groups were eliminated during the relational schema design process.

The following relations satisfy First Normal Form:

- USER
- DRIVER
- COMPANY
- END_CUSTOMER
- VEHICLE_CATEGORY
- VEHICLE
- TRUCK
- AGREEMENT
- SHIPMENT
- DRIVER_ASSIGNMENT
- INVOICE
- PAYMENT
- DELIVERY_VERIFICATION
- SHIPMENT_STATUS_LOG
- CANCELLATION_REQUEST

Therefore, the database satisfies First Normal Form (1NF).

3.3.2 Second Normal Form (2NF)

A relation is in Second Normal Form (2NF) if:

- It is already in First Normal Form.
- Every non-key attribute is fully functionally dependent on the entire primary key.

- No partial dependencies exist.

The VTMS database uses surrogate primary keys for all relations. Since every table contains a single-column primary key, each non-key attribute depends entirely on its respective primary key.

Examples of functional dependencies include:

$$company\_id \rightarrow company\_name,\ registration\_number,\ email,\ phone,\ address,\ wallet\_balance,\ status$$

$$vehicle\_id \rightarrow category\_id,\ registration\_number,\ make,\ model,\ year,\ color,\ status$$

$$shipment\_id \rightarrow company\_id,\ end\_customer\_id,\ vehicle\_id,\ agreement\_id,\ tracking\_id,\ shipment\_statu$$

Since no relation contains composite primary keys and no partial dependencies exist, all relations satisfy Second Normal Form (2NF).

3.3.3 Third Normal Form (3NF)

A relation is in Third Normal Form (3NF) if:

- It is already in Second Normal Form.
- No transitive dependency exists.
- Non-key attributes depend only on the primary key.

The VTMS database was analyzed to identify and remove transitive dependencies.

Initially, payment-related information could create a transitive dependency because shipment details and payment details were stored together. This could result in dependencies such as:

$$shipment\_id \rightarrow payment\_id$$

$$payment\_id \rightarrow amount,\ payment\_type,\ payment\_date,\ payment\_status$$

which produces the transitive dependency:

$$shipment\_id \rightarrow amount,\ payment\_type,\ payment\_date,\ payment\_status$$

To eliminate this dependency, payment information was separated into the PAYMENT relation and linked through foreign keys.

The resulting PAYMENT relation is:

$$PAYMENT(payment\_id,\ shipment\_id,\ company\_id,\ amount,\ payment\_type,\ payment\_date,\ paymen$$

This separation ensures that payment-related attributes depend directly on `payment_id` rather than indirectly through another non-key attribute.

Similarly, customer, company, vehicle category, invoice, and verification information are maintained in their respective relations and referenced using foreign keys, eliminating transitive dependencies throughout the database.

Therefore, all relations satisfy Third Normal Form (3NF).

3.3.4 Final 3NF Relations

After the normalization process, the following relations constitute the final normalized database schema of the Vehicle Transportation Management System.

- USER
- DRIVER
- COMPANY
- END_CUSTOMER
- VEHICLE_CATEGORY
- VEHICLE
- TRUCK
- AGREEMENT
- SHIPMENT

- DRIVER_ASSIGNMENT
- INVOICE
- PAYMENT
- DELIVERY_VERIFICATION
- SHIPMENT_STATUS_LOG
- CANCELLATION_REQUEST

The final database design satisfies First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF). The normalization process minimizes redundancy, eliminates update anomalies, improves data consistency, and ensures efficient database operations. “‘

CHAPTER 4 IMPLEMENTATION

This chapter describes the technologies, frameworks, and tools used to implement the Vehicle Transportation Management System (VTMS). It explains the development environment, application architecture, and the mechanisms used to connect the application with the database.

4.1 LANGUAGE / FRAMEWORK

The Vehicle Transportation Management System (VTMS) was developed using a modern full-stack web development architecture consisting of React.js for the frontend, Node.js with Express.js for the backend, and MySQL as the database management system.

4.1.1 Frontend Technology: React.js

React.js was selected for developing the graphical user interface (GUI) because it provides a component-based architecture that simplifies the development of complex user interfaces. React enables reusable UI components, efficient state management, and dynamic rendering of data.

The major advantages of React.js include:

- Fast and responsive user interfaces through Virtual DOM.
- Reusable components that improve maintainability.
- Easy integration with RESTful APIs.
- Efficient handling of dynamic data and state changes.
- Large community support and extensive documentation.

The frontend provides interfaces for:

- User authentication and login.
- Company management.
- Customer management.
- Shipment booking and tracking.
- Driver and truck assignment.
- Payment and invoice management.
- Delivery verification and shipment monitoring.

4.1.2 Backend Technology: Node.js and Express.js

Node.js was used to develop the server-side application logic, while Express.js was utilized as the web application framework.

Node.js was selected because it supports asynchronous and event-driven programming, making it highly suitable for handling multiple client requests efficiently.

Express.js provides a lightweight framework for developing RESTful APIs and managing server-side routing.

The major advantages include:

- High performance and scalability.
- Non-blocking I/O operations.
- Efficient handling of concurrent requests.
- Simple API development using Express.js.
- Easy integration with MySQL databases.

The backend is responsible for:

- Processing business logic.
- Managing user authentication and authorization.
- Handling CRUD operations.
- Managing shipment assignments.

- Processing payments and invoices.
- Communicating with external APIs.
- Maintaining database transactions.

4.1.3 Database Management System: MySQL

MySQL was selected as the relational database management system because it provides reliability, security, scalability, and excellent support for structured relational data.

The advantages of MySQL include:

- Strong support for relational database design.
- Enforcement of primary and foreign key constraints.
- Efficient query processing and indexing.
- ACID-compliant transaction management.
- Wide industry adoption and community support.

The database stores all system information including:

- Users and Drivers.
- Companies and End Customers.
- Vehicles and Trucks.
- Shipments and Assignments.
- Agreements.
- Invoices and Payments.
- Delivery Verification records.
- Shipment Status Logs.
- Cancellation Requests.

4.1.4 API Testing Tool: Thunder Client

Thunder Client was used for testing and validating REST API endpoints during development.

The tool provides:

- API request testing.
- JSON response validation.
- Authentication testing.
- CRUD operation verification.
- Endpoint debugging and troubleshooting.

4.1.5 External Service: OpenRouteService API

The OpenRouteService API was integrated to calculate transportation routes and distances between pickup and delivery locations.

OpenRouteService was selected because it provides free routing services suitable for academic and prototype projects, whereas commercial GPS and mapping APIs often impose usage limitations and associated costs.

The API is used for:

- Route calculation.
- Distance estimation.
- Shipment charge calculation.
- Transportation planning.

4.2 DATABASE CONNECTIVITY

The Vehicle Transportation Management System follows a three-tier architecture consisting of the Presentation Layer, Application Layer, and Database Layer.

4.2.1 System Architecture

The communication flow within the system is as follows:

1. The user interacts with the React.js frontend.
2. React sends HTTP requests to the Express.js REST API.
3. The Express.js server processes the request and executes the required business logic.
4. The backend communicates with the MySQL database.
5. Query results are returned to the backend.
6. The backend sends JSON responses to the frontend.
7. The frontend displays the processed information to the user.

4.2.2 Database Connection Technology

The Node.js backend establishes connectivity with MySQL using the MySQL2 package.

The MySQL2 library provides:

- Efficient database connectivity.
- Prepared statements.
- Connection pooling.
- Transaction support.
- Secure query execution.

4.2.3 Connection Configuration

The application uses the following database connection parameters:

- Host: localhost
- Port: 3306
- Database: VTMS

- Database Engine: MySQL
- Backend Runtime: Node.js
- Connection Library: MySQL2

The database credentials are stored securely using environment variables to prevent unauthorized access and improve security.

4.2.4 Connection Workflow

The following sequence is followed whenever the application accesses the database:

1. User submits a request through the React interface.
2. The request is sent to an Express.js API endpoint.
3. The controller validates the request data.
4. The service layer executes SQL queries through MySQL2.
5. MySQL processes the query and returns the result.
6. The backend converts the result into JSON format.
7. The JSON response is returned to the React frontend.
8. The frontend updates the user interface accordingly.

4.2.5 Security Measures

Several security mechanisms were implemented to protect database communication:

- Passwords are stored using hashing algorithms.
- SQL injection is prevented using parameterized queries.
- Environment variables are used to store database credentials.
- User authentication and role-based authorization are enforced.
- Database access is restricted to authorized backend services only.

The implemented architecture ensures reliable communication between the React frontend, Node.js backend, and MySQL database while maintaining performance, scalability, and data security.

4.3 STORED PROCEDURES AND FUNCTIONS

This section documents the database programmability objects developed for the Vehicle Transportation Management System (VTMS). These objects include Triggers and Stored Procedures that automate business rules, maintain data integrity, enforce constraints, and support operational workflows.

4.3.1 Triggers

- **trg_generate_tracking_id** (Trigger) Purpose: Auto-generates TRK-2026-XXXX tracking ID before inserting a shipment record. Input: Shipment Record Output: Tracking ID Generated
- **trg_generate_invoice_number** (Trigger) Purpose: Auto-generates INV-2026-XXXX invoice number before invoice creation. Input: Invoice Record Output: Invoice Number Generated
- **trg_calculate_delivery_charges** (Trigger) Purpose: Calculates delivery charges using distance, rate, multiplier and discount rules. Input: Shipment Data Output: Delivery Charges Calculated
- **trg_create_delivery_verification** (Trigger) Purpose: Creates blank delivery verification record after shipment creation. Input: Shipment ID Output: Verification Record Created
- **trg_prevent_status_regression** (Trigger) Purpose: Prevents shipment status from moving backward except cancellation. Input: Shipment Status Update Output: Validation Applied
- **trg_log_shipment_status** (Trigger) Purpose: Logs every shipment status change into ShipmentStatusLog. Input: Old/New Status Output: Log Entry Created
- **trg_update_vehicle_status** (Trigger) Purpose: Updates vehicle status based on shipment progress. Input: Shipment Update Output: Vehicle Status Updated
- **trg_update_driver_status** (Trigger) Purpose: Updates driver status during assignment lifecycle. Input: Driver Assignment Update Output: Driver Status Updated
- **trg_update_truck_status_on_assignment** (Trigger) Purpose: Updates truck status and load. Input: Assignment Update Output: Truck Status Updated
- **trg_auto_generate_invoice** (Trigger) Purpose: Auto-creates invoice when shipment is delivered. Input: Shipment Status Update Output: Invoice Created
- **trg_deduct_wallet_balance** (Trigger) Purpose: Deducts company wallet after payment completion. Input: Payment Record Output: Wallet Updated
- **trg_notify_admin_new_company** (Trigger) Purpose: Sends notification when new company registers. Input: Company Record Output: Notification Sent

4.3.2 Stored Procedures

- **sp_update_user_password** (Stored Procedure) Purpose: Updates password for system users. Input: user_id, new_hash Output: Password Updated
- **sp_update_company_password** (Stored Procedure) Purpose: Updates company password. Input: company_id, new_hash Output: Password Updated

- **sp_resolve_cancellation** (Stored Procedure) Purpose: Approves or rejects cancellation request. Input: request_id, approved_by, decision Output: Cancellation Processed

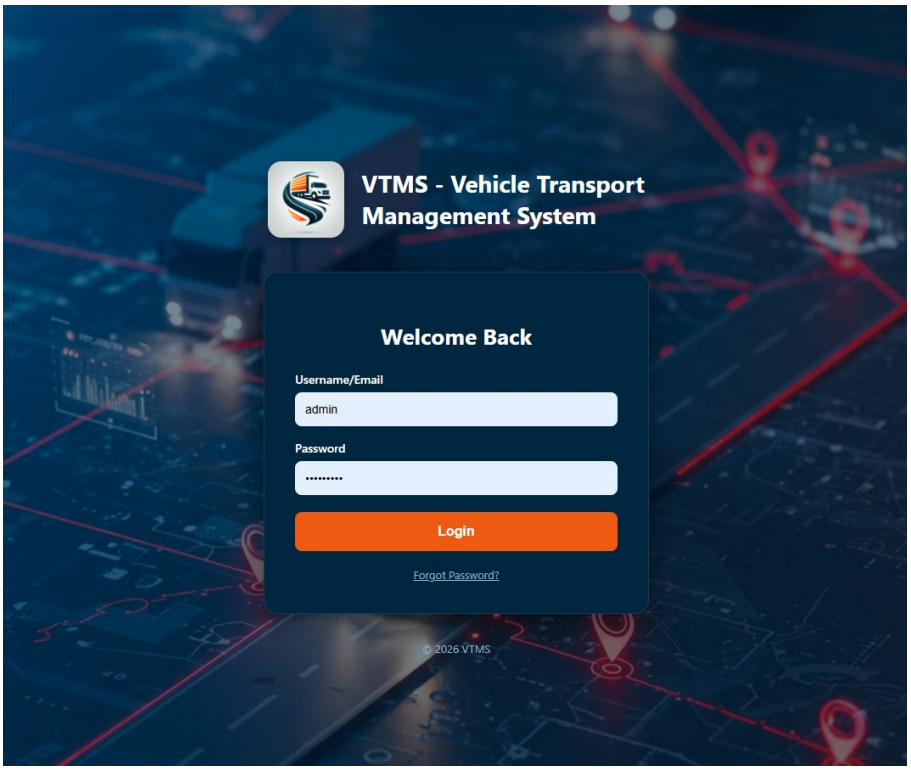
- **sp_approve_cod_payment** (Stored Procedure) Purpose: Approves COD payment. Input: payment_id, finance_manager_id Output: Payment Approved

4.4 INTERFACES

This section presents the graphical user interface (GUI) of the Vehicle Transportation Management System (VTMS). The system is developed using React.js and provides an interactive and user-friendly interface for managing shipments, users, vehicles, payments, and tracking operations.

Each interface plays a specific role in enabling efficient communication between the user and the system. The following screenshots illustrate the main functional modules of the system along with their descriptions.

4.4.1 Login Interface



The login interface allows users (Admin, Transport Manager, Finance Manager, and Company) to securely access the system. Authentication is performed using username and password with role-based redirection.

4.4.2 Admin Dashboard Interface

this interface allows Admin to manage whole system check Active shipments, Available Drivers and trucks, Manages shipments checks for the canceled orders manages users and deals with companies and checks the agreements

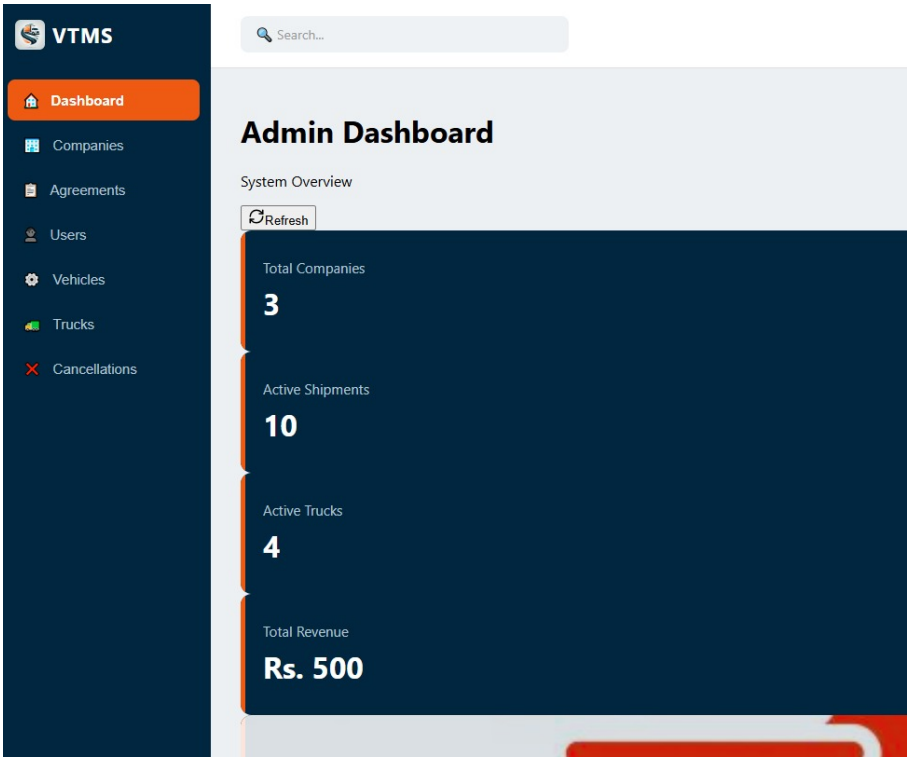


Figure 4.1: Admin Dashboard

4.4.3 Shipment Interface

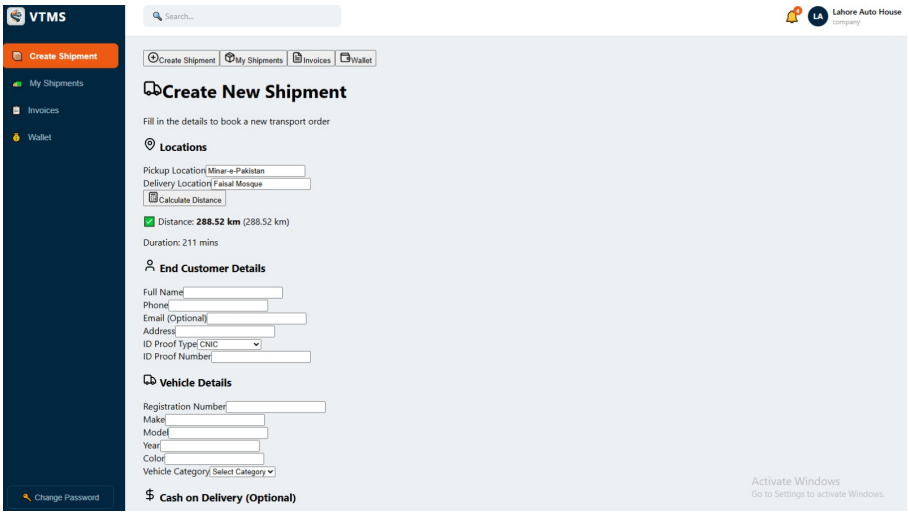


Figure 4.2: Shipment View

This interface allows company users to create new shipment requests by entering pickup location, delivery location, vehicle type, and shipment details. The system automatically calculates distance and charges using the OpenRouteService API.

4.4.4 Driver and Truck Assignment Interface

This interface is used by the Transport Manager to assign available drivers and trucks to shipments. The system ensures that only available resources are assigned.

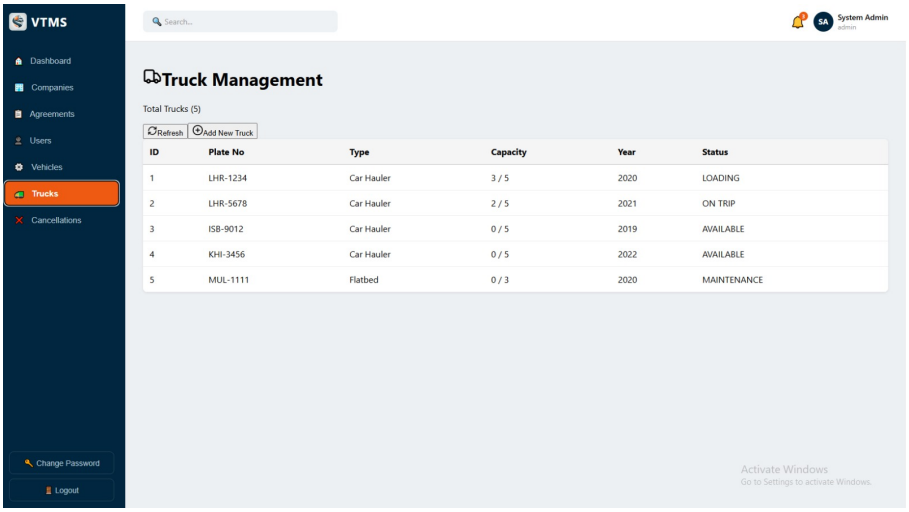


Figure 4.3: Truck Managment

4.4.5 Invoice Interface

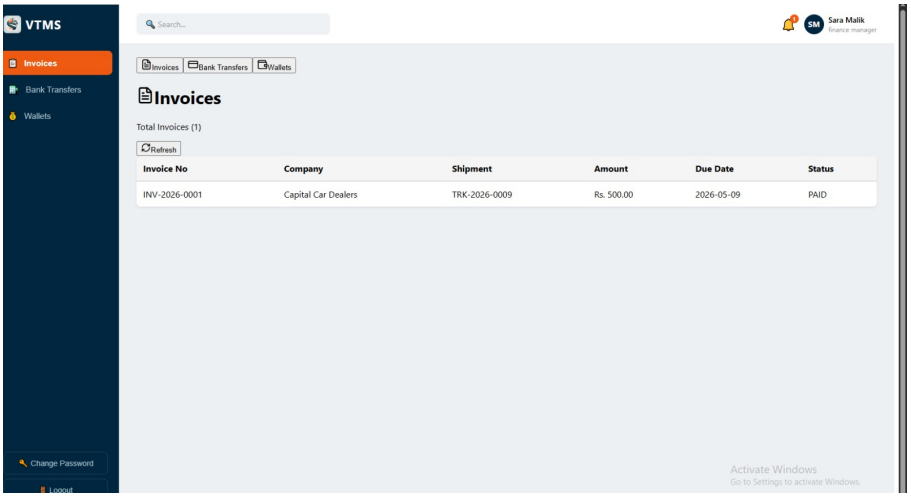


Figure 4.4: invoice Interface

The invoice interface displays automatically generated invoices for each completed shipment. It includes shipment details, total charges, and payment status.

REFERENCES

package (Node.js native binding): [Online]. Available: https://www.npmjs.com/package/bcrypt.

1. Google LLC, "Google Maps Platform — Distance Matrix and Directions API Documentation," [Online]. Available: https://developers.google.com/maps. [Accessed: April 2026].
2. Official docs: thunderclient [Online]. Available: https://docs.thunderclient.com/.
3. OpenJS Foundation, "Node.js v20 Documentation," [Online]. Available: https://nodejs.org/en/docs. [Accessed: April 2026].
4. Meta Platforms Inc., "React 18 Documentation," [Online]. Available: https://react.dev. [Accessed: April 2026].
5. openroutesourcehttps://openrouteservice.org/.
6. VTMS GitHub Repository, [Online]. Available: https://github.com/MominaUmar74/VTMS.git. [Accessed: April 2026].
7. OpenAI, "ChatGPT AI Assistant," Conversation Reference: https://chatgpt.com/c/69b25b22-37fc-83ab-bb9f-89a521259246. [Accessed: March–April 2026].

APPENDIX: AI USAGE AND PROMPTS

This section documents all use of AI tools during the development of the VTMS project. The table below lists each prompt submitted to an AI assistant, along with the specific purpose for which it was used.

AI Tool Used: ChatGPT (OpenAI)

Reference Conversation: `https://chatgpt.com/c/69b25b22-37fc-83ab-bb9f-89a521259246`

Sr.	Prompt Used	Purpose
1	We are building VTMS where companies register and make contracts with us. We deliver vehicles to end customers provided by the company. Design a complete database schema.	
2	Update the design: Company is main client, Agreement between Admin and Company, EndCustomer provided by Company, Driver as separate entity.	
3	Admin can create Transport Manager and Finance Manager. Transport Manager can create Driver. Show this hierarchy in ERD	
4	Add self-referencing in User table for reports_to. Driver should be separate entity.	
5	Design cancellation rules based on shipment status. OTP only for delivery verification, not for cancellation.	
6	Company pays after delivery. Support COD collection from end customer. Add Invoice and Company Wallet.	
7	7Generate full relational schema (MySQL) with User table containing Admin, TransportManager, FinanceManager and Driver as separate entity.	
8	Create Enhanced ERD in PlantUML with all entities, relationships, and self-referencing.	
9	Update the report sections (Entity, Business Rules) based on new Company + EndCustomer model	
10	.10Now based on revised rules update document. or ye prompts add kr do	

Table 4.1: AI Prompts Used and Their Purposes